Software Requirements Specification

Goal Tracker – Goal & Deadline Tracker

Native Android application written in Java

Package `com.example.goaltracker`

Source package:	`V3.4GoalTracker-native.zip`
Platform:	Android 8.0 (API 26) and later
Build system:	Shell scripts (`aapt2`, `javac`, `d8`, `apksigner`)
Document date:	October 17, 2026

This document was derived from the actual source files. Statements that could not be confirmed from the files are marked **[not verified]**.

Contents

1 Introduction **1**

1.1 Purpose . . . 1

1.2 Scope . . . 1

1.3 Definitions and Abbreviations . . . 2

1.4 Project Overview . . . 2

1.4.1 Product perspective . . . 2

1.4.2 Project facts established by analysis . . . 3

1.4.3 Document organisation . . . 3

1.5 Document Conventions . . . 3

2 Requirements **4**

2.1 Functional Requirements . . . 4

2.1.1 Goals . . . 4

2.1.2 Dates and validation . . . 4

2.1.3 Task lists and tasks . . . 5

2.1.4 Persistence, backup, export and import . . . 5

2.1.5 Appearance and widget . . . 6

2.2 Non-Functional Requirements . . . 7

3 System Requirements and Technology Stack **8**

3.1 Hardware and Software Requirements . . . 8

3.1.1 Target device (to run the app) . . . 8

3.1.2 Build machine . . . 8

3.2 Technology Stack and Dependencies . . . 9

3.3 Environment Variables Used by the Scripts . . . 10

4 System Architecture **11**

4.1 Overview . . . 11

4.2 Components . . . 11

4.3 Component Dependencies . . . 12

4.4 Key Runtime Flows . . . 12

4.4.1 Saving a change . . . 12

4.4.2 Start-up and recovery . . . 12
4.4.3 Widget interaction . . . 12
4.5 Android Manifest Summary . . . 13

5 Database, Local Data Storage, Import and Export 14
5.1 Storage Locations . . . 14
5.2 Document Format (schema version 2) . . . 14
5.3 Local Storage Folder . . . 15
5.4 Export . . . 15
5.5 Import . . . 16

6 User Interface and Application Features 17
6.1 Main Screen . . . 17
6.2 Task Editor . . . 17
6.3 Settings Screen . . . 17
6.4 Theming . . . 18
6.5 Home-Screen Widget . . . 18

7 Error Handling, Security, Testing and Validation 19
7.1 Error Handling and Data Validation . . . 19
7.2 Security and Data Integrity . . . 19
7.2.1 Findings . . . 19
7.2.2 Integrity measures . . . 20
7.3 Testing and Validation . . . 20

8 Setup Guide for a New Device 21
8.1 Prerequisites . . . 21
8.2 Step 1 – Install Termux . . . 21
8.3 Step 2 – Allow File Access (optional) . . . 21
8.4 Step 3 – Obtain the Source Code . . . 21
8.5 Step 4 – Install Build Tools (one time) . . . 22
8.6 Step 5 – Configure Environment (only if needed) . . . 22
8.7 Step 6 – Build the APK . . . 22
8.8 Step 7 – Install and Run . . . 22
8.9 Step 8 – Rebuilding After Changes . . . 23
8.10 Release Build . . . 23
8.11 Troubleshooting . . . 24

9 Terminal Command Reference **25**

9.1 Setup and Source Commands . . . 25

9.2 Build and Install Commands . . . 25

9.3 Environment Commands . . . 26

9.4 Repair Commands (only when a problem occurs) . . . 26

9.5 Individual Build Stages (what `build.sh` runs) . . . 27

9.6 Verification Commands (standard tools, optional) . . . 27

9.7 Commands That Do Not Apply . . . 28

9.8 Common Command Errors . . . 28

10 Project File and Dependency Reference **29**

10.1 Project Tree . . . 29

10.2 Source Code . . . 29

10.3 Configuration and Build Files . . . 30

10.4 Resources . . . 31

10.5 Generated and Binary Files . . . 31

10.6 Dependency Summary . . . 32

11 Known Limitations, Unverified Items and Future Enhancements **33**

11.1 Known Limitations (from the code and files) . . . 33

11.2 Inconsistencies Found Between Files . . . 34

11.3 Information That Could Not Be Verified . . . 34

11.4 Proposed Future Enhancements . . . 34

A Appendix: Quick Start Summary **35**

List of Figures

4.1 Component overview (arrows show the main direction of calls or data) 11

List of Tables

1.1 Definitions and abbreviations . . . 2
1.2 Facts established from the source package . . . 3

2.1 Functional requirements: goals . . . 4
2.2 Functional requirements: dates . . . 4
2.3 Functional requirements: lists and tasks . . . 5
2.4 Functional requirements: data handling . . . 6
2.5 Functional requirements: appearance and widget . . . 6
2.6 Non-functional requirements . . . 7

3.1 Runtime requirements . . . 8
3.2 Build-time requirements (as checked by build.sh) . . . 9
3.3 Technology stack . . . 9
3.4 Environment variables read by the scripts . . . 10

4.1 Dependencies between Java classes (from source references) . . . 12
4.2 Declared components . . . 13

5.1 Where data is stored . . . 14
5.2 Field meanings . . . 15

6.1 Palettes (from `Pal.java`) . . . 18

7.1 Validation and error handling found in the code . . . 19
7.2 Proposed manual acceptance checks . . . 20

8.1 Common problems (sources: README and script messages) . . . 24

9.1 Setup and source commands . . . 25
9.2 Build, install and run commands . . . 26
9.3 Environment commands . . . 26
9.4 Repair commands . . . 26
9.5 Optional verification commands . . . 27
9.6 Command errors and fixes . . . 28

10.1 Java source files (folder GoalTracker/src/com/example/goaltracker/) 29
10.2 Configuration, build and documentation files 30
10.3 Layouts and drawables 31
10.4 Generated artefacts included in the package 31
10.5 External dependencies 32

11.1 Documentation and comment inconsistencies (code was not changed) 34

Chapter 1

Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the Goal Tracker application as it exists in the supplied source package `V3.4GoalTracker-native.zip`. It is intended for developers who need to understand the system, set up a new build machine, compile the application and maintain it. Every requirement in Chapter 2 is traced to the source file in which the behaviour was found. Behaviour that could not be confirmed from the files is either omitted or marked **[not verified]**.

1.2 Scope

Goal Tracker is a single-user, offline Android application for tracking goals with deadlines. Each goal has task lists, and each list has tasks. The application also ships a home-screen widget that lists all goals with their remaining days and elapsed percentage. The scope of this document covers:

- the Android application and widget (`GoalTracker/src`, `GoalTracker/res`, `GoalTracker/AndroidManifest.xml`);
- the shell-based build tooling (`GoalTracker/setup.sh`, `GoalTracker/build.sh`);
- the data formats used for local storage, backup, export and import.

It does not cover any server, account system or network functionality, because none exists in the code.

Project type differs from common assumptions

This project is a **native Android app written in plain Java**. The analysis found *no* Flutter, Dart, Gradle, Android Studio project files, AndroidX or third-party libraries. The README states the same. Consequently, no Flutter, Dart or Gradle commands appear in this document. The build is performed by `build.sh`, which calls `aapt2`, `javac`, `d8` and `apksigner` directly.

1.3 Definitions and Abbreviations

Table 1.1: Definitions and abbreviations

Term	Meaning
SRS	Software Requirements Specification
APK	Android application package, the installable file
API level	Android platform version number (minimum 26, target 34 in this project)
SAF	Storage Access Framework: Android's folder and file picker mechanism
Termux	Linux-style terminal environment for Android, used as the primary build host
aapt2	Android Asset Packaging Tool: compiles and links resources and the manifest
d8	Dexer converting Java `.class` files into `classes.dex`
apksigner	Tool that signs an APK
Widget	Home-screen app widget (`AppWidgetProvider`)
Store	The in-memory data model and persistence class, `Store.java`
Goal / List / Task	The three levels of the data hierarchy

1.4 Project Overview

1.4.1 Product perspective

The application is self-contained. It uses only classes from the Android framework (`android.*`), `org.json` and the Java standard library (`java.time`, `java.util`, `java.io`). It declares one permission, `RECEIVE_BOOT_COMPLETED`, so the widget can be refreshed after a reboot. No internet permission is declared in the manifest.

1.4.2 Project facts established by analysis

Table 1.2: Facts established from the source package

Item	Value (source)
Package name	com.example.goaltracker (`AndroidManifest.xml`)
App label	"Goal Tracker" (`AndroidManifest.xml`)
minSdk / targetSdk	26 / 34 (manifest and `build.sh`)
Version code / name	1 / 1.0, passed to `aapt2 link` in `build.sh`
Language	Java; compiled with `javac --release 8`
Java source files	11 files, 2 637 lines in total (`src/com/example/goaltracker/`)
Largest file	`MainActivity.java`, 875 lines
XML resources	2 layouts, 11 drawables, 2 adaptive-icon files, 2 values folders (day/night), 1 widget definition
Activities	`MainActivity`, `SettingsActivity`, `WidgetMenuActivity`
Other components	`GoalWidgetProvider` (receiver), `GoalWidgetService` (service)
Unit or instrumented tests	None found in the package
Version control data	No `.git` directory in the package; a `GoalTracker.gitignore` file exists
Pre-built artefacts included	`GoalTracker/GoalTracker.apk` and the `GoalTracker/build/` directory

1.4.3 Document organisation

Chapter 2 lists requirements. Chapters 3–7 explain the technology, architecture, data and user interface. Chapters 8 and 9 give the new-device setup guide and the command reference. Chapter 10 documents every file. Chapter 11 lists limitations, unverifiable items and proposed enhancements.

1.5 Document Conventions

Requirement identifiers use the prefixes FR (functional) and NFR (non-functional). The source column names the file in which the behaviour was found. Commands are shown in code blocks and are labelled **Required** or **Optional**.

Chapter 2

Requirements

2.1 Functional Requirements

2.1.1 Goals

Table 2.1: Functional requirements: goals

ID	Requirement	Source
FR-G1	The app shall support multiple goals, shown as horizontally scrolling tabs, with a "+ New goal" control.	`MainActivity.buildTabs`
FR-G2	A new goal shall be created titled "New goal", with start date today, deadline today + 30 days and one empty list named "To do".	`Store.addGoal`
FR-G3	The goal title shall be editable inline and saved on every change.	`MainActivity.inlineField`
FR-G4	A goal shall be deletable after a confirmation dialog ("Delete this goal and all its lists?").	`MainActivity`, `Store.deleteGoal`
FR-G5	The user shall be able to choose a deadline and a start date with the native date picker.	`MainActivity.pickDeadline`, `pickStart`
FR-G6	The goal card shall show days remaining, deadline date, total days, "remaining / total – %" and a live d/h/m/s countdown refreshed every second.	`MainActivity.buildHero`, `ticker`
FR-G7	An overall progress bar and the text "N% · X/Y tasks done" shall be shown.	`MainActivity`, `BarView`
FR-G8	On first launch with no stored data, a sample goal "Build an Android App" (two lists, four tasks) shall be created.	`Store.seed`

2.1.2 Dates and validation

Table 2.2: Functional requirements: dates

ID	Requirement	Source
FR-D1	A deadline earlier than the start date shall be rejected with the message "Invalid deadline. The deadline cannot be earlier than the start date."	`Store.setDeadline`, `MSG_INVALID_DEADLINE`
FR-D2	The deadline picker shall disable days before the start date.	`MainActivity.pickDate` (`setMinDate`)

ID	Requirement	Source
FR-D3	If a new start date falls after the current deadline, the new start shall be held in memory only until a valid deadline is chosen; an invalid pair shall never be stored.	`Store.setStart`, `Goal.pendingStart`
FR-D4	Total duration shall be stored with the goal and recalculated only when start or deadline changes.	`Goal.recalcTotal`
FR-D5	Remaining days shall use calendar dates (not hours), so results do not depend on time of day or zone.	`Util.daysRemaining`
FR-D6	Elapsed percentage = (total−remaining)/total×100, clamped to 0–100; 99% maximum while days remain; 100% at or after the deadline; no division by zero.	`Util.percentElapsed`

2.1.3 Task lists and tasks

Table 2.3: Functional requirements: lists and tasks

ID	Requirement	Source
FR-T1	Each goal shall contain any number of lists; lists can be added ("+ New list"), renamed inline and deleted.	`Store.addList`, `deleteList`
FR-T2	A task shall have name, description, start date, optional due date, priority (High/Medium/Low), status (Incomplete/Completed) and progress.	`Store.Task`, `openTaskForm`
FR-T3	Task progress shall be set with a slider in 5% steps (0–100).	`MainActivity` (`seek.setMax(20)`)
FR-T4	Setting a task to Completed shall set progress to 100; toggling the checkbox sets `done` and progress 100 or 0.	`Store.toggleTask`
FR-T5	Tasks shall be reorderable (up/down), editable and deletable.	`Store.moveTask`, `upsertTask`, `deleteTask`
FR-T6	A task whose due date has passed and which is not done shall be shown as overdue.	`Task.overdue`
FR-T7	A goal's percentage shall be the rounded mean of its tasks' effective progress (0 if there are no tasks).	`Goal.percent`

2.1.4 Persistence, backup, export and import

Table 2.4: Functional requirements: data handling

ID	Requirement	Source
FR-S1	Every change shall be saved automatically to `SharedPreferences` (file `goal_tracker`, key `goal-tracker-v1`).	`Store.persist`
FR-S2	Each save shall also write atomic backups `latest.json` and `last-nonempty.json` in `files/backup`.	`Store.backup`, `writeFile`
FR-S3	If primary data is missing or unreadable, the app shall recover from the newest valid backup; unreadable data is kept as `corrupt-<time>.json`.	`Store.load`
FR-S4	Data of an older schema shall be copied to `pre-migration-v<N>.json` before being rewritten in schema 2.	`Store.load`
FR-S5	A "Restore from backup" button shall replace current data with the latest backup after confirmation.	`MainActivity.addRestoreButton`
FR-S6	The user may choose a storage folder via the folder picker; the app maintains `GoalTracker/goal-tracker-data.json` there.	`LocalFolder`, `SettingsActivity`
FR-S7	If the chosen folder already contains data, the user shall choose between using it and overwriting it.	`SettingsActivity.afterInspect`
FR-S8	Export shall write all data to a user-chosen JSON file and verify it by reading it back.	`SettingsActivity.doExport`
FR-S9	Import shall validate the file, then offer Replace or Merge (Merge adds only goals with unseen ids).	`SettingsActivity.doImport`, `Store.importDocument`
FR-S10	A safety copy `pre-import-<time>.json` shall be written before each import; only the newest five are kept.	`Store.pruneSafetyCopies`

2.1.5 Appearance and widget

Table 2.5: Functional requirements: appearance and widget

ID	Requirement	Source
FR-A1	Settings shall offer Light, Dark and System default appearance; the choice is stored in `goal_tracker_settings`.	`Pal`, `SettingsActivity`
FR-A2	Changing appearance shall re-theme the screens and refresh the widget.	`SettingsActivity.choose`
FR-W1	The widget shall display a scrollable list of all goals ordered by soonest deadline, with active goals first, then completed or expired, then goals without a deadline.	`GoalWidgetProvider.rank`

ID	Requirement	Source
FR-W2	Each row shall show the goal name, "remaining / total" days and "N% complete", "100% · Expired" or "Completed".	`GoalWidgetService`
FR-W3	Tapping a row shall open that goal; "+" shall open the app and add a goal; tapping the empty view adds a goal.	`GoalWidgetProvider`, `MainActivity.handleWidgetIntent`
FR-W4	The "• • •" button shall open a popup with Open Goal Tracker, Add new goal, Settings and Refresh widget.	`WidgetMenuActivity`
FR-W5	The widget shall refresh on every save, at midnight, on date/time/time-zone change, on boot and after an app update.	`GoalWidgetProvider.onReceive`

2.2 Non-Functional Requirements

Table 2.6: Non-functional requirements

ID	Requirement	Evidence
NFR-1	Runs on Android 8.0 (API 26) or newer.	Manifest `minSdkVersion`
NFR-2	Works fully offline; no network permission is declared.	Manifest
NFR-3	No dependency on third-party libraries or AndroidX.	`build.sh`, imports
NFR-4	Saves must not leave half-written backup files (temp file then rename, with `sync`).	`Store.writeFile`
NFR-5	Folder syncing shall not block the UI: writes run on a single background thread and only the newest data is written.	`LocalFolder`
NFR-6	Imported files larger than 10 MB shall be refused.	`LocalFolder.MAX_BYTES`
NFR-7	Layout adapts to wide screens (content width capped at 900 dp, settings at 640 dp).	`MainActivity`, `SettingsActivity`
NFR-8	Dates use the device calendar date so results are time-zone independent.	`Util`
NFR-9	The APK shall be buildable on a phone (Termux) without Gradle or an IDE.	`README.md`, `build.sh`

Chapter 3

System Requirements and Technology Stack

3.1 Hardware and Software Requirements

3.1.1 Target device (to run the app)

Table 3.1: Runtime requirements

Item	Requirement
Operating system	Android 8.0 (API 26) or newer
Storage	About 71 KB for the APK (size of the supplied `GoalTracker.apk`) plus user data
Permissions	`RECEIVE_BOOT_COMPLETED` only; no storage permission (folder access uses the system picker)
Install source	"Install unknown apps" must be allowed for the installing app (Files app or Termux)

3.1.2 Build machine

The documented build host is a phone running **Termux** (README, `setup.sh`). `setup.sh` uses Termux's `pkg` package manager and therefore only works inside Termux. `build.sh` is a Bash script that needs the tools listed below; running it on a desktop Linux system with the same tools on the `PATH` is plausible but **[not verified]**, because the supplied files only document Termux.

Table 3.2: Build-time requirements (as checked by build.sh)

Tool	Termux package	Role
`javac`, `keytool`	`openjdk-17`	Compile Java; create signing key
`aapt2`	`aapt2`	Compile/link resources and manifest
`d8`	`d8`	Convert classes to DEX
`apksigner`	`apksigner`	Sign the APK
`zip`	`zip`	Add `classes.dex` to the APK
`zipalign`	`zipalign` (optional)	Align APK; skipped if missing
`curl`, `unzip`	`curl`, `unzip`	Used by `setup.sh` and for extraction
`android.jar`	downloaded by `setup.sh`	Android API stubs for compilation
`adb`	`android-tools` [**not verified**]	Optional, for `--install`

The README states that setup needs internet access and roughly 300 MB of storage. The name of the Termux package that provides `adb` is not given in the project files.

3.2 Technology Stack and Dependencies

Table 3.3: Technology stack

Layer	Technology
Language	Java (source level 8 through `--release 8`; code uses lambdas, try-with-resources and `java.time`)
UI	Programmatic Android Views (no XML layouts for the main screens); XML layouts only for the widget
Framework	Android framework APIs only (`Activity`, `AppWidgetProvider`, `RemoteViewsService`, `DocumentsContract`, `SharedPreferences`)
JSON	`org.json` (bundled with Android)
Dates	`java.time.LocalDate` (available from API 26)
Build	Shell scripts: `aapt2` → `javac` → `d8` → `zip` → `apksigner`
Third-party libraries	None
Package manager files	None (no `build.gradle`, `pubspec.yaml`, `pom.xml` or `package.json`)

The manifest targets API 34, but `setup.sh` downloads `android.jar` for API 33 first, falling back to 32, 31, 30, 29 and 28. The code is therefore compiled against stubs of API 28–33. `java.time` usage requires API 26 or later, which matches `minSdkVersion`.

3.3 Environment Variables Used by the Scripts

Table 3.4: Environment variables read by the scripts

Variable	Default	Used by
`ANDROID_JAR`	`$HOME/android-sdk/android.jar`	`setup.sh`, `build.sh`
`KEYSTORE`	`$HOME/.android/goaltracker.keystore`	`build.sh`
`JAVA_TOOL_OPTIONS`	not set; README suggests `-Xmx512m`	JVM (troubleshooting)

Chapter 4

System Architecture

4.1 Overview

The app has a single process with one shared data object (`Store`). All screens, the widget and the background folder sync read from or write through that object.

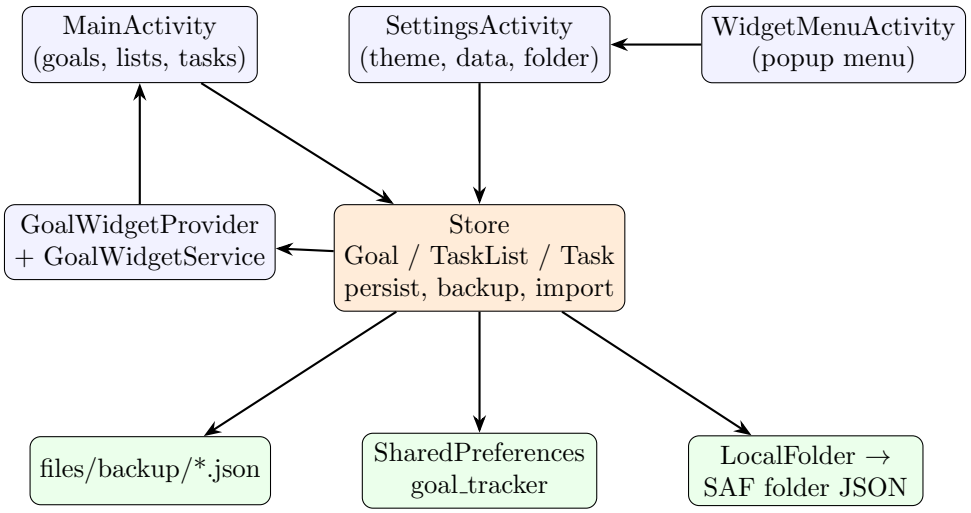


Figure 4.1: Component overview (arrows show the main direction of calls or data)

4.2 Components

MainActivity Builds the entire main screen in code. It re-renders from the store whenever the store calls its `listener`, and runs a 1-second ticker while resumed to update the countdown. It also receives widget intents (extras `widget_add_goal`, `widget_goal_id`).

SettingsActivity Appearance choice, storage folder, sync, export and import.

WidgetMenuActivity A transparent activity drawn as a dropdown, opened by the widget button.

Store Singleton (`Store.get(context)`) holding goals, active goal id, models, persistence, backup, recovery, import and date rules.

LocalFolder Optional mirror of the data into a user-selected folder using `DocumentsContract`.

GoalWidgetProvider / GoalWidgetService Widget logic; the service supplies the scrollable rows.

Util, Pal Date helpers; colour palette and saved appearance mode.

BarView, FlowLayout Small custom views (progress bar, wrapping row layout).

4.3 Component Dependencies

Table 4.1: Dependencies between Java classes (from source references)

Class	Uses
MainActivity	Store, Pal, Util, BarView, FlowLayout, GoalWidgetProvider, R resources
SettingsActivity	Store, LocalFolder, Pal, Util, GoalWidgetProvider
WidgetMenuActivity	Pal, Util, MainActivity, SettingsActivity, GoalWidgetProvider
Store	Util, LocalFolder, GoalWidgetProvider, `org.json`
LocalFolder	Android `DocumentsContract`, `ContentResolver`
GoalWidgetProvider	Store, Pal, GoalWidgetService, MainActivity, WidgetMenuActivity
GoalWidgetService	GoalWidgetProvider, Pal
Pal	Android resources (`R.style`)
Util	`java.time`

4.4 Key Runtime Flows

4.4.1 Saving a change

1. A UI action calls a `Store` method (for example `toggleTask`).
2. `Store.changed()` calls `persist()`, then runs the UI listener.
3. `persist()` builds JSON, checks that it parses and contains `goals`, commits it to `SharedPreferences`, writes the backups, queues a folder sync, and refreshes the widget.

4.4.2 Start-up and recovery

`Store.get()` loads `SharedPreferences`. If the data parses, it is used (after a pre-migration copy when the schema is older than 2). Otherwise the raw text is stored as a `corrupt-*.json` file, the latest valid backup is restored, and as a last resort the sample goal is created.

4.4.3 Widget interaction

The widget list is served by `GoalWidgetService`, which reads the live `Store`. Row taps carry a fill-in intent with the goal id to `MainActivity`. Because the store may be edited while the service reads it, `loadItems` retries up to three times on a runtime exception.

4.5 Android Manifest Summary

Table 4.2: Declared components

Component	Exported	Notes
MainActivity	yes	Launcher; `singleTask`; `adjustResize`
SettingsActivity	no	–
WidgetMenuActivity	no	Translucent `MenuTheme`; no history; excluded from recents
GoalWidgetProvider	yes	Receives widget update, date/time/zone change, boot, package-replaced and a custom midnight action
GoalWidgetService	no	Requires `BIND_REMOTEVIEWS`

The widget metadata (`res/xml/goal_widget_info.xml`) sets a minimum size of 250×220 dp, a target of 4×3 cells, `updatePeriodMillis` of 1 800 000 (30 minutes) and resizing in both directions.

Chapter 5

Database, Local Data Storage, Import and Export

The application uses no SQL database. All data is a single JSON document.

5.1 Storage Locations

Table 5.1: Where data is stored

Location	Name	Content
SharedPreferences file	`goal_tracker`, key `goal-tracker-v1`	Primary copy of all data (JSON string)
App files directory	`files/backup/latest.json`	Latest valid document
	`files/backup/last-nonempty.json`	Latest valid document that has at least one goal
	`pre-migration-v<N>.json`	Copy of old-schema data before migration
	`corrupt-<millis>.json`	Unreadable primary data preserved for inspection
	`pre-import-<millis>.json`	Safety copy before an import (newest 5 kept)
SharedPreferences file	`goal_tracker_settings`	`theme_mode` and folder-sync keys (`folder_tree_uri`, `folder_file_uri`, `folder_label`, `folder_last_sync`, `folder_error`)
User-chosen folder (optional)	`GoalTracker/goal-tracker-data.json`	Mirror of the data, rewritten after each save

5.2 Document Format (schema version 2)

```
{
  "v": 2,
  "active": "k3j9x0a",
  "goals": [
    {
      "id": "k3j9x0a", "title": "Build an Android App",
```

```
      "deadline": "2026-01-15", "created": "2025-10-22", "total": 85,
      "lists": [
        { "id": "p2m1q8z", "name": "Design",
          "tasks": [
            { "id": "a1b2c3d", "name": "Sketch main screens",
              "desc": "Wireframes", "start": "2025-10-22",
              "due": "2025-10-29", "priority": "high",
              "progress": 100, "done": true }
          ] }
      ]
    }
  ]
}
```

Listing 5.1: Structure of the saved JSON document (values are examples)

Table 5.2: Field meanings

Field	Meaning
`v`	Schema version (constant `SCHEMA = 2`); version 1 had no stored `total`
`active`	Id of the selected goal, or `null`
`created`	The goal's *start date* (key name kept for compatibility)
`deadline`	ISO date `yyyy-MM-dd`
`total`	Whole days from start to deadline, never negative
`priority`	`high`, `medium` (default) or `low`
`progress`, `done`	Task progress 0–100 and completion flag; a done task counts as 100
ids	7 random base-36 characters (`Util.newId`)

Missing fields fall back to defaults when parsing (for example priority `medium`, progress 0).

5.3 Local Storage Folder

The folder is chosen with `ACTION_OPEN_DOCUMENT_TREE`. The app takes a persistable read/write grant, looks for or creates a `GoalTracker` subfolder and `goal-tracker-data.json`, and writes with mode `"wt"` on a single-thread executor. A pending-write holder keeps only the newest JSON. A failed write is retried once with a fresh lookup; if it still fails an error text is stored and shown in Settings. *Stop syncing* clears the stored keys and releases the permission but leaves existing files.

5.4 Export

Export data uses `ACTION_CREATE_DOCUMENT` with the suggested name `goal-tracker-backup-<date>.json`. After writing, the file is read back and parsed; the number of goals must match, otherwise "The exported file could not be verified" is shown.

5.5 Import

1. The user selects a file (`ACTION_OPEN_DOCUMENT`).
2. The file is read (maximum 10 MB, an optional UTF-8 byte-order mark is removed) and parsed without changing app state.
3. Errors shown: “That file is not a Goal Tracker backup” or “That backup contains no goals”.
4. A dialog states the number of goals and tasks and offers **Replace**, **Merge** or Cancel.
5. A `pre-import-*.json` safety copy is written, then data is applied. In Merge mode goals whose id already exists are skipped. Duplicate ids inside the file receive a new id. A goal whose deadline is before its start has its start set to the deadline.

Chapter 6

User Interface and Application Features

6.1 Main Screen

The main screen is built in `MainActivity.render()` and consists of, from top to bottom:

1. **Goal tabs** (scrolling chips, a "+ New goal" item and a settings icon at the right).
2. **Goal card** with editable title, Start and Deadline buttons, a "Delete goal" button, the statistics (Days remaining, Deadline date, Total days, Remaining / total – %, Time left d/h/m/s) and the overall progress bar. On wide screens the five statistics share one row; otherwise they wrap into three rows.
3. **Task lists** heading with "+ New list".
4. **List cards**: inline-editable name, delete control and the tasks with priority pill, dates, progress, checkbox, up, down, edit and delete controls.
5. A **Restore from backup** button at the bottom.

When there are no goals, "No goals yet" with a hint is shown. When a goal has no lists, "No lists yet. Create one to start adding tasks." is shown.

6.2 Task Editor

A dialog titled "Add task" or "Edit task" contains: Task name (max 120 characters), Description (max 400), Start and Due dates (due may be cleared), Priority (High / Medium / Low), Status (Incomplete / Completed) and a progress slider. Status and slider are linked: choosing Completed sets 100%, and moving the slider changes the status.

6.3 Settings Screen

- **Appearance**: Light, Dark, System default (radio list).
- **Data & storage**: Storage folder, Sync now and Stop syncing (shown only when a folder is set), Export data, Import data.

6.4 Theming

Table 6.1: Palettes (from `Pal.java`)

Role	Light	Dark
Background	`#FFFFFF`	`#000000`
Card	`#F5F5F5`	`#111111`
Text	`#000000`	`#FFFFFF`
Muted text	`#6B6B6B`	`#A0A0A0`
Lines	`#D4D4D4`	`#2E2E2E`

The palette is black and white only; priority and overdue states are expressed through contrast rather than hue. Themes `AppTheme`, `AppTheme.Light`, `AppTheme.Dark` and `MenuTheme` are defined in `res/values/styles.xml`.

6.5 Home-Screen Widget

The widget layout (`res/layout/goal_widget.xml`) has a "+" and a "• • •" button above a `ListView`. Rows (`res/layout/widget_row.xml`) show the name, a vertical separator, and "remaining / total" with a status line. To add it: long-press the home screen, choose Widgets, then *Goal Tracker*. In System mode colours come from resources that differ by day and night; in Light or Dark mode, colours are set from code.

Chapter 7

Error Handling, Security, Testing and Validation

7.1 Error Handling and Data Validation

Table 7.1: Validation and error handling found in the code

Situation	Behaviour	Source
Deadline before start	Rejected; alert "Invalid date" and picker re-opens	`Store`, `MainActivity`
Start after deadline	Start kept in memory only; user must choose a new deadline	`Store.setStart`
Unparseable date	Parsing returns `null`; UI shows "No date" or "Not set"; calculations treat it as missing	`Util`
Corrupt primary data	Raw text saved as `corrupt-*.json`; restore from backup; else sample data	`Store.load`
Invalid import file	Toast with reason; no state change	`SettingsActivity`
Oversized file	`IOException` "File is too large" (10 MB)	`LocalFolder.readAll`
Folder access revoked	Error stored; Settings shows "Sync problem" text	`LocalFolder.flush`
No file/folder picker	Toast "No folder picker is available on this device"	`SettingsActivity`
Widget read during edit	Up to three retries, then an empty list	`GoalWidgetProvider.loadItems`
Text length	Goal and list field filters; task name 120, description 400	`MainActivity`

7.2 Security and Data Integrity

7.2.1 Findings

- **Local only:** no network permission and no network code were found.
- **Private storage:** primary and backup data are in app-private storage (`MODE_PRIVATE`, `getFilesDir()`).
- **Exports and the sync folder are plain, unencrypted JSON** in locations the user chooses.
- **Folder access** uses scoped persistable URI grants, not broad storage permissions.

- **Exported components:** only `MainActivity` (launcher) and `GoalWidgetProvider` are exported. The widget service is protected by `BIND_REMOTEVIEWS`.
- **PendingIntents:** the widget uses `FLAG_IMMUTABLE` for fixed intents and `FLAG_MUTABLE` for the row template and menu intent, which need the launcher to add data.
- `android:allowBackup="true"` is set, so the system backup mechanism may include app data **[not verified]** (behaviour depends on the device).

7.2.2 Integrity measures

Validation before saving (round-trip parse), temp-file-and-rename writes with `sync`, a non-empty backup that is never overwritten by an empty one, schema versioning with pre-migration copies, and safety copies before import.

Signing key

`build.sh` creates a keystore with the fixed passwords `android` (store and key) and a 10 000-day validity. This is acceptable for personal builds but not for public distribution. The same key must be reused for updates: an APK signed with a different key cannot update an installed copy, and the user must uninstall first, which deletes app data unless it was exported.

7.3 Testing and Validation

No test sources, test frameworks or CI configuration exist in the package, so no automated test results can be claimed. The following manual checks are derived from the requirements and are proposals, not recorded results.

Table 7.2: Proposed manual acceptance checks

ID	Check	Req.
T1	Fresh install shows the sample goal “Build an Android App”.	FR-G8
T2	Pick a deadline before the start: alert appears and nothing is saved.	FR-D1
T3	Move the start after the deadline: “Select deadline” prompt appears.	FR-D3
T4	Complete a task: progress shows 100% and the goal percentage updates.	FR-T4, T7
T5	Force-stop and relaunch: all data is still present.	FR-S1
T6	Export, then Import with Replace and with Merge: counts match the dialogs.	FR-S8, S9
T7	Set a storage folder, edit a goal, open the JSON file in the folder.	FR-S6
T8	Add the widget; verify rows, “+”, “• • •” menu and row taps.	FR-W1–W4
T9	Switch Light/Dark/System: app and widget change.	FR-A1, A2
T10	Change the device date or time zone: widget day counts refresh.	FR-W5

Build validation: a successful `bash build.sh` prints `SUCCESS: ...GoalTracker.apk`; `apksigner verify` can additionally be run (optional, see Chapter 9).

Chapter 8

Setup Guide for a New Device

This guide follows the build path documented by the project (README, `setup.sh`, `build.sh`): **building on an Android phone or tablet with Termux**. No desktop IDE is needed. Commands are listed with purpose and expected output in Chapter 9.

Other build hosts

The project files do not describe building on Windows, macOS or desktop Linux. `setup.sh` depends on Termux's `pkg`. A desktop build would need the same tools installed by other means and a Bash shell; this is **[not verified]** and not covered here.

8.1 Prerequisites

- An Android device (the build host) with about 300 MB free storage and internet access for setup.
- The source archive `V3_4GoalTracker-native.zip` on that device (for example in Downloads).
- A device to run the app: Android 8.0 or later. It may be the same device.

8.2 Step 1 – Install Termux

Install Termux from **F-Droid** (`https://f-droid.org/packages/com.termux/`). The README states that the Play Store version is outdated and will not work. Open Termux once to let it finish initialising.

8.3 Step 2 – Allow File Access (optional)

Run `termux-setup-storage` and accept the prompt. This creates `~/storage/downloads`, from which the archive is read and to which `build.sh` copies the finished APK. Without it the build still works, but the APK stays inside Termux's private folder.

8.4 Step 3 – Obtain the Source Code

No remote repository address is contained in the package, so the source is obtained from the archive only. Extract it in the Termux home directory:

```
cd ~
unzip ~/storage/downloads/V3_4GoalTracker-native.zip
```

The archive contains a top-level folder `V3.4GoalTracker-native/` with the project in `V3.4GoalTracker-native/GoalTracker/`. The README assumes an archive named `GoalTracker-native.zip` that extracts directly to `GoalTracker/`; adjust the path to your file name and folder layout:

```
cd ~/V3.4GoalTracker-native/GoalTracker
ls
```

You should see `AndroidManifest.xml`, `build.sh`, `setup.sh`, `src` and `res`.

8.5 Step 4 – Install Build Tools (one time)

```
bash setup.sh
```

The script (a) updates and upgrades Termux packages, (b) installs `openjdk-17 aapt2 apksigner d8 zip unzip curl`, (c) tries to install `zipalign` (optional), (d) checks that `curl` works, and (e) downloads `android.jar` to `~/android-sdk/android.jar` by trying API 33 down to 28. Success ends with "Setup complete. Now run: bash build.sh". Re-running is safe: an existing `android.jar` is kept.

8.6 Step 5 – Configure Environment (only if needed)

No configuration is needed for a normal first launch. Optional overrides:

```
export ANDROID_JAR=/path/to/android.jar     # use a different API stub jar
export KEYSTORE=/path/to/my.keystore        # use a different signing key
export JAVA_TOOL_OPTIONS=-Xmx512m           # if Java runs out of memory
```

Variables last for the current Termux session only. If `KEYSTORE` points to an existing file, its store and key password must both be `android`, because `build.sh` passes that value.

8.7 Step 6 – Build the APK

```
bash build.sh
```

The script prints seven stages (clean, compile resources, link, compile Java, DEX, package, sign). On the first run it creates the keystore `~/.android/goaltracker.keystore`. The result is `GoalTracker.apk` in the project folder and, if storage access was granted, a copy in `~/storage/downloads/`.

8.8 Step 7 – Install and Run

1. Run `bash build.sh --install`, or open `Downloads/GoalTracker.apk` in the Files app.
2. On the first install, Android asks to allow the installing app to "install unknown apps"; enable it and tap Install.
3. Open *Goal Tracker* from the launcher. A fresh install shows the sample goal.
4. Optional: add the widget (long-press home screen → Widgets → Goal Tracker).

If an older copy signed with a different key exists, uninstall it first (export data beforehand).

8.9 Step 8 – Rebuilding After Changes

Edit files under `src/` or `res/`, then run `bash build.sh` again. The script wipes `build/` each time. Source code and the pre-built `GoalTracker.apk` in the package are not modified by reading this document.

8.10 Release Build

There is no separate debug or release configuration. `build.sh` always runs `d8 --release` and signs the APK with the local key; the manifest does not set `android:debuggable`. For personal use this APK is the release artefact. Publishing to a store would require a properly protected keystore and non-default passwords, which the script does not support without editing it **[not verified]** (store submission is not covered by the project files).

8.11 Troubleshooting

Table 8.1: Common problems (sources: README and script messages)

Problem	Solution
`Missing tool 'd8'` (or `aapt2`, `javac`, `keytool`, `apksigner`, `zip`)	Run `bash setup.sh` again, or `pkg install d8 aapt2 apksigner openjdk-17 zip`.
`android.jar not found at ...`	Run `bash setup.sh`.
`android.jar` download fails	Copy any `platforms/android-XX/android.jar` with XX $\geq$ 26 from an Android SDK to `~/android-sdk/android.jar`, then run `bash build.sh`.
`curl is broken` message from `setup.sh`	Run `termux-change-repo` and select a mirror, then `pkg upgrade -y` and `apt install --reinstall -y libc++ openssl libcurl curl aapt2`.
A required package failed to install	Re-run `bash setup.sh` or try `pkg upgrade -y` first.
"App not installed"	Install from **Downloads** rather than Termux's private folder or a path with square brackets; uninstall an older copy first; turn off Samsung Auto Blocker. For the exact error, pair wireless debugging, run `adb connect localhost:PORT`, then `bash build.sh --install`.
`zipalign` missing	The script skips alignment. In the supplied APK, `resources.arsc` was inspected and is stored uncompressed and 4-byte aligned, which is what Android 11+ requires for apps targeting API 30 or higher, so skipping `zipalign` was fine for that build (see Chapter 11).
Out of memory in `javac` or `d8`	Close other apps; `export JAVA_TOOL_OPTIONS=-Xmx512m`.
`No such file` for the zip	Check the archive name and the extra top-level folder (Step 3).
Compiler errors after your edits	Read the `javac` message printed in stage 4; stage numbers help locate which tool failed.

Chapter 9

Terminal Command Reference

All commands run in the **Termux** app on Android unless noted. **Required** means needed for the documented build; **Optional** means helpful but not needed. Commands marked "standard tool" are not called by the project scripts; they are common usage of the named tool and were *not* executed during this analysis (the analysis environment had no Android toolchain or network).

9.1 Setup and Source Commands

Table 9.1: Setup and source commands

Command	Status	Prerequisite	Purpose / expected result
`termux-setup-storage`	Optional	Termux installed	Shows a permission prompt; afterwards `~/storage/downloads` exists. Needed to read the zip from Downloads and to get the APK copied there.
`cd ~`	Required	–	Moves to the Termux home directory.
`unzip <path-to-zip>`	Required	`unzip` installed (preinstalled or via `pkg install unzip`)	Extracts the project into `V3.4GoalTracker-native/`. Error "cannot find zipfile" means the path or file name is wrong.
`cd ~/V3.4GoalTracker-native/GoalTracker`	Required	Archive extracted	Enters the folder containing `build.sh`.
`ls`	Optional	–	Confirms `AndroidManifest.xml`, `build.sh`, `setup.sh`, `src`, `res` are present.
`bash setup.sh`	Required (once)	Internet, about 300 MB	Installs JDK 17, `aapt2`, `apksigner`, `d8`, `zip`, `unzip`, `curl`; downloads `android.jar`. Ends with "Setup complete". See Table 9.6 for errors.

9.2 Build and Install Commands

Table 9.2: Build, install and run commands

Command	Status	Prerequisite	Purpose / expected result
`bash build.sh`	Required	`setup.sh` done	Seven stages, then "SUCCESS: .../GoalTracker.apk (size)". Can be run from any directory because the script changes into its own folder.
`bash build.sh --install`	Optional	Successful build prerequisites	Builds, then installs: through `adb install -r` if a device is attached in `adb devices`, else `termux-open` opens the installer.
`termux-open GoalTracker.apk`	Optional	APK built; `termux-open` present in Termux	Opens Android's package installer for the APK.
`cp GoalTracker.apk ~/storage/downloads/`	Optional	Storage access granted	Manual copy so the Files app can install it (`build.sh` already does this when possible).
`adb connect localhost:PORT`	Optional	Wireless debugging paired; `adb` installed	Lets `--install` print the real install error. `PORT` is shown by the Android developer settings.

9.3 Environment Commands

Table 9.3: Environment commands

Command	Status	Purpose
`export ANDROID_JAR=/path/android.jar`	Optional	Overrides the API stub jar used by both scripts.
`export KEYSTORE=/path/my.keystore`	Optional	Overrides the signing keystore (passwords must be `android`).
`export JAVA_TOOL_OPTIONS=-Xmx512m`	Optional	Limits JVM memory when `javac` or `d8` run out of memory.

9.4 Repair Commands (only when a problem occurs)

Table 9.4: Repair commands

Command	Status	Purpose
`pkg install d8 aapt2 apksigner openjdk-17 zip`	Optional	Installs the missing build tools by hand.

Command	Status	Purpose
`pkg upgrade -y`	Optional	Fixes mixed package versions.
`termux-change-repo`	Optional	Chooses a different package mirror.
`apt install --reinstall -y libc++ openssl libcurl curl aapt2`	Optional	Repairs a broken `curl` as suggested by `setup.sh`.

9.5 Individual Build Stages (what `build.sh` runs)

For understanding or debugging, these are the exact tool invocations inside `build.sh`. Running `bash build.sh` is the supported way; do not run these by hand unless debugging. `$B` is `GoalTracker/build`.

```
aapt2 compile --dir res -o $B/res.zip
aapt2 link -o $B/base.apk -I $ANDROID_JAR --manifest AndroidManifest.xml \
  --java $B/gen --min-sdk-version 26 --target-sdk-version 34 \
  --version-code 1 --version-name 1.0 --auto-add-overlay $B/res.zip
javac --release 8 -Xlint:-options -classpath $ANDROID_JAR -d $B/obj \
  $(find src $B/gen -name '*.java')
d8 --release --lib $ANDROID_JAR --min-api 26 --output $B/dex \
  $(find $B/obj -name '*.class')
cp $B/base.apk $B/unaligned.apk
(cd $B/dex && zip -q -j $B/unaligned.apk classes.dex)
zipalign -f -p 4 $B/unaligned.apk $B/aligned.apk     # only if zipalign exists
keytool -genkeypair -keystore $KEYSTORE -alias goaltracker -keyalg RSA \
  -keysize 2048 -validity 10000 ...                  # only if keystore is missing
apksigner sign --ks $KEYSTORE --ks-pass pass:android --key-pass pass:android \
  --out GoalTracker.apk $B/aligned.apk
```

Listing 9.1: Tool commands issued by build.sh (abridged to the essential arguments)

9.6 Verification Commands (standard tools, optional)

Table 9.5: Optional verification commands

Command	Status	Purpose
`apksigner verify --verbose GoalTracker.apk`	Optional, standard tool	Checks the APK signature.
`adb devices`	Optional, used by `build.sh`	Lists attached devices; a line ending in `device` enables adb install.

9.7 Commands That Do Not Apply

Note

There is no project-specific use of `flutter`, `dart`, `gradle`, `./gradlew`, `npm` or `sdkmanager`. No unit-test command exists because the package has no tests. No Git repository metadata is included, so no `git clone` URL can be given. If you want version control for your own copy, `git init` followed by copying `GoalTracker.gitignore` to `.gitignore` (it ignores `build/`, `*.apk`, `*.idsig`) is a reasonable standard practice but is not part of the project.

9.8 Common Command Errors

Table 9.6: Command errors and fixes

Message	Command	Fix
`Missing tool '...'`	`build.sh`	Run `bash setup.sh`.
`android.jar not found`	`build.sh`	Run `bash setup.sh` or set `ANDROID_JAR`.
"Could not download android.jar"	`setup.sh`	Copy an `android.jar` (API $\geq$ 26) to `~/android-sdk/android.jar`; re-run the build.
"A required package failed to install"	`setup.sh`	Re-run, or `pkg upgrade -y`.
`curl` broken message	`setup.sh`	`termux-change-repo`, then the `apt install --reinstall` command above.
`bash: build.sh: No such file`	`bash build.sh`	You are in the wrong folder; `cd` into `GoalTracker/`.
Java `OutOfMemoryError`	`javac`, `d8`	`export JAVA_TOOL_OPTIONS=-Xmx512m`.
"App not installed"	installer	See Chapter 8 troubleshooting.

Chapter 10

Project File and Dependency Reference

Paths are relative to `V3.4GoalTracker-native/`. Type: **Logic**, **Config**, **Resource**, **Build**, **Doc** or **Artefact** (generated).

10.1 Project Tree

```
V3.4GoalTracker-native/
|-- GoalTracker.gitignore
‘-- GoalTracker/
    |-- AndroidManifest.xml
    |-- README.md
    |-- setup.sh
    |-- build.sh
    |-- GoalTracker.apk              (pre-built, signed)
    |-- src/com/example/goaltracker/ (11 Java files)
    |-- res/
    |   |-- drawable/  (11 XML)      |-- layout/ (2 XML)
    |   |-- mipmap-anydpi-v26/ (2)   |-- xml/ (1)
    |   |-- values/ (2)              ‘-- values-night/ (2)
    ‘-- build/                       (generated by build.sh)
```

Listing 10.1: Directory layout of the supplied package (76 files)

10.2 Source Code

Table 10.1: Java source files (folder GoalTracker/src/com/example/goaltracker/)

File (lines)	Type	Purpose and responsibilities	Depends on
`MainActivity.java` (875)	Logic	Launcher activity. Builds tabs, goal card, lists, task rows and task editor dialog in code; date pickers; live countdown ticker; restore button; handles widget intents.	Store, Pal, Util, BarView, FlowLayout, GoalWidgetProvider
`Store.java` (510)	Logic	Models `Task`, `TaskList`, `Goal`; singleton store; JSON load/save; migration; backup and recovery; import; date rules; CRUD operations.	Util, LocalFolder, GoalWidgetProvider, org.json

File (lines)	Type	Purpose and responsibilities	Depends on
`SettingsActivity.java` (402)	Logic	Settings: appearance, storage folder, sync, export, import and their dialogs.	Store, LocalFolder, Pal, Util
`LocalFolder.java` (201)	Logic	Optional mirror folder through `DocumentsContract`; background writer; shared read helper with size limit.	Android SAF classes
`GoalWidgetProvider.java` (187)	Logic	Widget provider: builds `RemoteViews`, intents, midnight alarm, item ordering, refresh and event handling.	Store, Pal, GoalWidgetService, MainActivity, WidgetMenuActivity
`WidgetMenuActivity.java` (122)	Logic	Popup menu activity opened from the widget.	Pal, Util, MainActivity, SettingsActivity
`Util.java` (91)	Logic	Date parsing and formatting, day and percent calculations, id generation, dp conversion.	java.time
`Pal.java` (78)	Logic	Palettes and saved theme mode; status-bar styling; theme selection.	R.style
`GoalWidgetService.java` (73)	Logic	`RemoteViewsService` supplying widget rows.	GoalWidgetProvider, Pal, `widget_row.xml`
`FlowLayout.java` (63)	Logic (view)	Wrapping layout for chips and pills.	Android views
`BarView.java` (35)	Logic (view)	Rounded progress bar drawn on a canvas.	Android graphics

10.3 Configuration and Build Files

Table 10.2: Configuration, build and documentation files

File	Type	Purpose and contents
`GoalTracker/AndroidManifest.xml`	Config	Package, SDK levels, boot permission, three activities, widget receiver, widget service.
`GoalTracker/build.sh`	Build	7-stage build and signing script; variables `ANDROID_JAR`, `KEYSTORE`; optional `--install`.
`GoalTracker/setup.sh`	Build	Termux tool installation and `android.jar` download.
`GoalTracker/README.md`	Doc	Features, layout, Termux build steps, troubleshooting, differences from a Flutter predecessor.
`GoalTracker.gitignore`	Config	Ignore rules: `build/`, `*.apk`, `*.idsig`.
`res/xml/goal_widget_info.xml`	Config	Widget size, resize mode, 30-minute period, initial layout.

File	Type	Purpose and contents
`res/values/styles.xml`, `res/values-night/styles.xml`	Resource	Themes `AppTheme`, `AppTheme.Light`, `AppTheme.Dark`, `MenuTheme`.
`res/values/colors.xml`, `res/values-night/colors.xml`	Resource	`app_bg`, `icon_bg` and widget colours `w_*` for day and night.

10.4 Resources

Table 10.3: Layouts and drawables

File	Purpose
`res/layout/goal_widget.xml`	Widget frame: "+" and "• • •" buttons, `ListView`, empty-state text.
`res/layout/widget_row.xml`	One goal row in the widget.
`res/drawable/widget_bg.xml`, `widget_bg_light.xml`, `widget_bg_dark.xml`	Rounded widget backgrounds (resource-colour, forced light, forced dark).
`res/drawable/ic_back,ic_close,ic_delete,ic_down,ic_edit,ic_event,ic_settings,ic_up.xml`	Vector icons used by the Java UI code.
`res/drawable/ic_add.xml`	Vector icon; no reference was found in code or XML.
`res/drawable/widget_btn_bg.xml`	Drawable; no reference was found in code or XML.
`res/drawable/ic_launcher_fg.xml`	Launcher icon foreground.
`res/mipmap-anydpi-v26/ic_launcher.xml`, `ic_launcher_round.xml`	Adaptive launcher icons (background `icon_bg`, foreground `ic_launcher_fg`).

10.5 Generated and Binary Files

Table 10.4: Generated artefacts included in the package

File	Description
`GoalTracker/GoalTracker.apk`	Final signed APK (71 463 bytes). Contains the manifest, resources, `classes.dex` and a signature.
`GoalTracker/build/res.zip`	Compiled resources (`aapt2 compile`).

File	Description
`GoalTracker/build/base.apk`	Linked resources and manifest (`aapt2 link`).
`GoalTracker/build/gen/.../R.java`	Generated resource id class.
`GoalTracker/build/obj/**/*.class`	Compiled classes, including nested classes such as `Store$Goal` and `GoalWidgetService$Factory`.
`GoalTracker/build/dex/classes.dex`	DEX bytecode from `d8`.
`GoalTracker/build/unaligned.apk`, `aligned.apk`	Intermediate APKs (identical size, since `zipalign` was skipped).

All items under `build/` are recreated by `build.sh`, which deletes the directory at the start.

10.6 Dependency Summary

Table 10.5: External dependencies

Dependency	Where it comes from
Android framework classes	`android.jar` at build time; the device at run time
`org.json`	Part of the Android platform
Java standard library	JDK 17 compiles with `--release 8`
Third-party libraries	None
Package-manager manifests	None

Chapter 11

Known Limitations, Unverified Items and Future Enhancements

11.1 Known Limitations (from the code and files)

1. **Single device, no cloud sync.** Data lives on the device; the folder sync and export are manual or local only.
2. **Unencrypted data.** Exports and the sync folder are plain JSON.
3. **No automated tests** and no CI configuration are included.
4. **Fixed signing passwords.** `build.sh` uses `android` for keystore and key passwords; a custom keystore must use the same.
5. **Android only.** The README notes the earlier Flutter app's iOS widget is not included.
6. **Flutter data is not migrated** (different storage location) although the JSON structure is the same, as stated in the README.
7. **Widget limits.** Android widgets cannot show true popups, so the menu is a separate activity. The system limits how often `updatePeriodMillis` refreshes; the app compensates with explicit refreshes and a midnight alarm (inexact, `setAndAllowWhileIdle`).
8. **Single list order and no search or filter** for tasks: none was found in the code.
9. **Task dates are not validated against each other** in the checks inspected (only goal start/deadline have a validation rule). **[not verified]** for the task editor in full.
10. **Unused resources:** `ic_add.xml` and `widget_btn_bg.xml` are not referenced.

11.2 Inconsistencies Found Between Files

Table 11.1: Documentation and comment inconsistencies (code was not changed)

Where	Observation
`setup.sh` comment	Says "targetSdk 29 needs no alignment", but `build.sh` targets 34. The shipped APK's `resources.arsc` is nevertheless uncompressed and 4-byte aligned, so it meets the Android 11+ requirement.
`GoalWidgetProvider` class comment	Mentions a "View all goals" button; the layout and code have "+" and "• • •" only. The README confirms the menu replaced it.
README step 3	Assumes an archive that extracts to `GoalTracker/` with the name `GoalTracker-native.zip`; the supplied archive has another name and an extra top-level folder.
README wording	Describes parity with a Flutter app; the Flutter sources are not in the package.
`Util.java`, `Store.java` comments	Mention ports of Dart/Flutter classes; they do not affect behaviour.

11.3 Information That Could Not Be Verified

- Whether the project builds on desktop Linux, macOS or Windows.
- Real execution of the build: the analysis machine had no Android toolchain or network. The supplied APK and `build/` files were inspected but `build.sh` was not re-run.
- Runtime behaviour on specific launchers and Android versions beyond what the README states.
- A remote source repository and release history (none in the package).
- Termux package name for `adb`.

11.4 Proposed Future Enhancements

These are suggestions only and are not implemented.

- Unit tests for `Util` (percent and day calculations) and `Store` (parse, import, migration), runnable on a desktop JVM.
- Configurable keystore passwords in `build.sh` and a documented release procedure.
- Optional encrypted export.
- Search, filter and sort for tasks; reminders or notifications for deadlines.
- A desktop-friendly build path and documented Linux/macOS setup.
- Splitting `MainActivity.java` (875 lines) into smaller view-builder classes.
- Removing unused resources and stale comments.

Appendix A

Appendix: Quick Start Summary

For a machine that already has Termux (F-Droid) and the archive in Downloads:

```
termux-setup-storage
cd ~
unzip ~/storage/downloads/V3_4GoalTracker-native.zip
cd ~/V3.4GoalTracker-native/GoalTracker
bash setup.sh          # once
bash build.sh --install
```

Source of facts

Every statement in this document comes from these files: the manifest, `build.sh`, `setup.sh`, `README.md`, `GoalTracker.gitignore`, all files under `src/` and `res/`, and inspection of the shipped APK. No source file was modified.

Bibliography

[1] Android Developers. *Android API reference and App Widgets guide*. https://developer.android.com/

[2] Android Developers. *Android Asset Packaging Tool 2 (aapt2)*. https://developer.android.com/tools/aapt2

[3] Android Developers. *apksigner*. https://developer.android.com/tools/apksigner

[4] Android Developers. *d8 dexer*. https://developer.android.com/tools/d8

[5] Termux project. *Termux terminal emulator for Android*. https://termux.dev/ and https://f-droid.org/packages/com.termux/

[6] Android Developers. *Storage Access Framework / DocumentsContract*. https://developer.android.com/guide/topics/providers/document-provider

[7] Project file `GoalTracker/README.md` (bundled with the source).